

FORWARD DEPLOYED ENGINEER

Take-Home Assignment

Build and deploy “The Lenny Growth Assistant”

Welcome

Thank you for your interest in the Forward Deployed Engineer role. This assignment evaluates how you turn an ambiguous business problem into a working, deployable AI product. We are looking for someone who can move comfortably between customer needs, product decisions, software engineering, AI systems, and operational handoff.

The strongest submissions will not only work technically; they will show clear judgment about what to build, what to simplify, how to communicate trade-offs, and how another team could run and extend the solution.

Engagement scenario

A product and growth team has asked you to turn Lenny’s Podcast transcripts into a reliable internal assistant. Their users want grounded answers, reusable written content, and rendered artifacts—without needing to understand prompts, models, or infrastructure. You are responsible for shaping the requirements, building the solution, deploying it locally, and handing it over in a form the client can evaluate.

1. Objective

Build a full-stack, AI-powered conversational web application called “The Lenny Growth Assistant.” The application must ingest transcripts from Lenny’s Podcast, answer complex product and growth questions, generate highly formatted content grounded in that knowledge, and create Markdown or HTML/CSS artifacts that render natively inside the product.

Treat this as a small forward-deployment engagement—not only a coding exercise. Make reasonable assumptions, document them, and optimize for an evaluator who needs to understand, run, test, and trust your system quickly.

2. Forward Deployment Brief

Before implementation, include a short discovery brief in your PRD covering:

- **User and problem:** Who is the primary user, what job are they trying to complete, and what pain does the assistant remove?
- **Success metric:** Define at least one measurable product or operational success metric.
- **Assumptions:** Record the important assumptions you made because the client brief was incomplete.
- **Scope choices:** Clearly state what you included, what you intentionally excluded, and why.

- **Risks and trade-offs:** Identify key risks such as hallucination, latency, cost, local-model quality, data leakage, or unsafe artifact rendering.

3. Core Requirements

3.1 API, sessions, and persistence

- **Backend framework:** Build the backend API using FastAPI.
- **Agent integration:** Build the agent layer using the Anthropic Claude Agent SDK or Pi Coding Agent.
- **Session handling:** Users must be able to start a new chat. Each session must maintain independent context.
- **Persistence:** Store conversations, session IDs, timestamps, and user metadata in PostgreSQL. You may use Supabase or Railway.
- **API quality:** Define clear request/response contracts, validation, structured errors, and health endpoints.

3.2 Flexible LLM configuration

Create a configuration layer that allows the evaluator to switch the underlying model without changing application code:

- **Cloud LLM:** Integrate at least one cloud provider, such as Anthropic Claude or OpenAI.
- **Local LLM—mandatory for the demo:** Run the submitted demo using Ollama and a model that works comfortably on your machine.
- **Toggle behavior:** Make the selected provider visible in the UI or configuration and document fallback behavior.

3.3 Knowledge base

Data source: Use the transcripts from [Lenny's Podcast / Newsletter transcript repository](#).

- **Ingestion:** Explain how transcripts are loaded, chunked or selected, indexed, refreshed, and traced back to their source.
- **Grounding:** Answers must cite or clearly identify the relevant transcript/source used.

4. Product Tasks

4.1 Grounded conversational assistant

Implement a RAG or long-context system that answers product management and growth questions strictly from Lenny's transcripts. The experience should handle follow-up questions, preserve session context, and acknowledge when the available material does not support an answer.

4.2 Ship 30 for 30 content skill

Create a dedicated skill or tool that turns grounded answers into a Ship 30 for 30–style essay. Read the linked source, identify the relevant writing principles, and encode them in the skill rather than relying on an unstructured one-off prompt.

- Approximately 1,250 words
- A strong hook and clear narrative progression
- Skimmable formatting with headings, bullets, and selective bold emphasis
- A specific, useful takeaway
- Claims grounded in the transcript knowledge base

4.3 Artifact generation and in-app viewer

When requested, the assistant must generate Markdown documents or complete HTML/CSS snippets based on the current conversation. The frontend must include an Artifact Viewer, similar to Claude Artifacts, that renders the result beside the chat instead of displaying only raw code or redirecting to another application.

Security expectation

Treat generated HTML as untrusted. Explain and implement a reasonable isolation or sanitization strategy for artifact rendering. The evaluator should be able to understand what the viewer permits, blocks, and why.

5. Deployment & Operational Readiness

A Forward Deployed Engineer must leave behind a solution another team can operate. Include:

- **One-command startup:** Provide a practical setup path, ideally using Docker Compose or an equivalent reproducible workflow.
- **Configuration:** Supply a .env.example with safe defaults and clear required/optional variables. Never commit secrets.
- **Observability:** Add structured logs and enough visibility to diagnose model, retrieval, database, and artifact-rendering failures.
- **Resilience:** Handle missing keys, unavailable Ollama, model timeouts, empty retrieval results, and database connection failures gracefully.
- **Handoff:** Document how a client engineer can run, test, troubleshoot, and extend the system.

6. Required Deliverables

#	Deliverable	What to include
1	Public GitHub repository	Complete source code with a sensible project structure and no committed secrets.
2	README.md	Architecture overview, prerequisites, installation, environment variables, local and cloud model setup, run commands, tests, and troubleshooting.
3	PRD	User, problem, success metric, assumptions, scope, flows, acceptance criteria, risks, and implementation plan.
4	design.md	Your UI/UX principles, information architecture, key interaction states, responsive behavior, accessibility considerations, and design decisions.
5	architecture.md	Database schema, API endpoints, component boundaries, ingestion/retrieval flow, agent routing, model toggle, security, and deployment topology.
6	Agent transcripts	Include coding-agent transcripts/logs in a dedicated folder, including failed attempts and how you corrected them. Remove secrets and sensitive data before committing.
7	Tests	Include meaningful automated tests for critical API, retrieval, routing, and persistence behavior, plus a short manual test plan for the UI.

#	Deliverable	What to include
8	Demo video	Record a 2–3 minute video with your camera enabled. Explain the problem, show the product, demonstrate local Ollama, and briefly cover one important technical trade-off. Upload it to YouTube.

7. Evaluation Criteria

- **Customer & product judgment:** Quality of discovery framing, assumptions, prioritization, success metrics, and trade-off decisions.
- **Technical execution:** End-to-end functionality across the UI, FastAPI, PostgreSQL, agent layer, retrieval, and model configuration.
- **Agentic architecture & grounding:** Clear skill boundaries, reliable routing, source-grounded answers, and sensible failure behavior.
- **Deployment & operability:** Reproducibility, observability, resilience, security, documentation, and evaluator handoff.
- **Code quality:** Separation of concerns, readability, maintainability, validation, error handling, and meaningful tests.
- **UI/UX quality:** A polished chat experience, understandable states, useful artifact viewer, responsive layout, and accessibility.
- **Communication:** Clarity of the PRD, architecture, design rationale, README, demo, and explanation of decisions.

8. Helpful Resources

- [FastAPI](#)
- [Ollama](#)
- [Anthropic Claude Agent SDK](#)
- [Pi Coding Agent](#)
- [Supabase](#)
- [Railway](#)
- [Ship 30 for 30 guide](#)
- [Impeccable](#)

Use AI tools thoughtfully

You are encouraged to use coding agents such as Claude, Codex, Cursor, or Devin. We are evaluating your judgment and ability to direct, verify, and improve AI-assisted work—not whether every line was typed manually.

Good luck—we are excited to see how you approach the deployment.